BUG-FIX AGENT

Team Execution Guide

One plan for four people, AI agents, Git branches, testing, and a clean GitHub merge

Project in one sentence

When a GitHub issue gets the agent-fix label, the system reads it, reproduces the bug in Docker, writes a failing test, diagnoses and fixes the bug, verifies red-to-green, and opens a draft pull request. A human must review and merge it.

Team member	Main responsibility
You (project lead / QA)	QA, demo repo, tickets and acceptance checks, release coordination, pitch, final demo
Subhi	Frontend only: Next.js pages, dashboard, components, API/SSE integration, frontend tests
Durgesh	AI agent: LangGraph pipeline, prompts, LLM calls, cache, red-to-green logic, PR explanation
Yash	Backend and infrastructure: FastAPI, data/auth, Docker sandbox, GitHub App/webhooks, deployment plumbing

This role assignment is a practical proposal based on the names and Subhi's frontend-only limit. The project files define four role tracks but do not assign names to those tracks.

PLAIN ENGLISH • 24–48 HOUR HACKATHON • ₹0 MVP BUDGET

1. Understand what we are building

The product is a bug-fix agent for a prepared Python repository. A developer performs one action: add the label agent-fix to a GitHub issue. The system should then run this sequence:

Step	What happens	Proof / result
1. Read	Read issue title, body, repository and reproduction steps.	A run is created; issue status comment says picked up.
2. Reproduce	Clone and test the repo inside an isolated Docker container.	A new test fails before the fix.
3. Diagnose	Find the likely cause using the issue and test evidence.	Plain-English root cause.
4. Fix	Make the smallest useful code change.	Patch is created on an agent branch.
5. Verify	Run the reproduction test and full existing pytest suite.	New test passes and existing suite stays green.
6. Draft PR	Push branch and open a draft pull request.	PR includes red/green proof, root cause and confidence.

Non-negotiable product rules

MVP is Python + pytest only. Use the prepared seeded demo repository. Run clone/install/test inside Docker. After dependency install, the test container has no network. No host mounts. Run as non-root with CPU, memory and time limits. Never put tokens inside the sandbox. The agent opens draft PRs only; it never merges.

What success looks like

- Label three seeded GitHub issues during the demo.
- At least two issues produce draft PRs with a failing-before / passing-after test and root-cause notes.
- The third issue honestly ends with a “couldn’t reproduce” comment. Do not invent a fix.
- The dashboard shows all three runs and their timeline end to end.

2. Who owns what

One person owns each area. Other people can review or help, but do not edit the same files at the same time unless the owner agrees.

You - project lead, QA, demo and release

- Create the GitHub project board and break work into small tickets from 11-tickets.md. Keep each ticket small enough for one focused AI-agent session and one commit.
- Create and maintain the seeded Python demo repo with three deterministic bugs: off-by-one, wrong condition, and missing null check. Keep known expected behaviour and a manual reproduction test for each.
- Write the acceptance checklist for each ticket. Check that the code matches the product brief, does not claim auto-merge, and does not exceed Python/pytest MVP scope.
- Own QA coordination: run or coordinate pytest, Vitest, integration tests, Playwright, axe accessibility, visual checks, Lighthouse and manual device tests.
- Own the demo script, slide deck, recorded backup run, rehearsal and judge-facing explanation. Emphasize red-to-green proof, Docker isolation, mandatory human review and ₹0 scope.
- Coordinate launch: collect sign-offs, verify smoke tests, run backup/restore and rollback drills, and keep the team focused on the critical path.

Tools: GitHub Projects / Issues / Pull Requests, seeded Python repo, pytest, Playwright, axe-playwright, Lighthouse CI, browser devtools, Android Chrome, iOS Safari, Google Slides or PowerPoint, screen recording tool.

Subhi - frontend only

- Set up and own `apps/web/` using Next.js App Router. Do not build FastAPI, Docker, LangGraph, webhook processing or OAuth token exchange.
- Build the design system: CSS variables, Tailwind theme, responsive layout, light/dark mode, reusable button/card/skeleton/empty-state/toast/status-badge components.
- Build the marketing pages: `/`, `/how-it-works`, `/demo`, `/security`, `/pricing`, `/about`, `/changelog`, `/privacy`, `/terms`.
- Build the frontend auth routes and screens, but call the backend auth endpoints. Do not store client secrets in the browser.
- Build `/dashboard` and `/dashboard/runs/[runId]`: run list, six-step timeline, red/green test output, root cause, confidence and draft PR link.
- Implement `lib/api-client.ts`, `lib/session.ts` and `lib/sse.ts`; handle loading, empty, error, partial, offline, signed-out and no-permission states.
- Write Vitest + React Testing Library tests, Playwright E2E tests for frontend journeys, axe scans and visual snapshots. Fix frontend bugs found by QA.

Tools: VS Code / Cursor / Codex / Claude / Open Code / Anti Gravity (choose one main editor-agent); Node.js LTS; npm; Next.js; TypeScript; Tailwind; Vitest; React Testing Library; Playwright; axe-playwright; Lighthouse CI; browser devtools; Vercel.

Durgesh - AI agent and LangGraph

- Own `apps/api/pipeline/langgraph.py` and prompt-related files. Agree on API and shared data shapes with Yash before building.
- Build the six nodes in order: Read, Reproduce, Diagnose, Fix, Verify, PR. Keep each node small and testable.
- Define a typed graph state: run ID, repo and issue details, working branch, reproduction test, before output, root cause, patch, after output, existing-suite output, confidence and errors.
- Write prompts that ask for evidence, a minimal patch, and honest uncertainty. Treat issue text and repository content as untrusted data, not as instructions to reveal secrets or escape limits.
- Use the sandbox interface from Yash. First prove a reproduction test fails before changing product code. Then patch and prove it passes.
- Do not open a PR unless the reproduction test passes after the fix and the full existing pytest suite remains green.
- Generate the PR body: what broke, why, what changed, failing-before/passing-after output, full-suite result, confidence High/Medium/Low and reasons.
- Add checkpoint/retry handling and make failures end in a clear status such as could_not_reproduce or failed. Never invent a successful run.

Tools: Python, LangGraph, Pydantic/shared schemas, pytest, chosen LLM free tier (Gemini or Groq / hackathon credits), HTTP client or provider SDK, hashing/cache helper, GitHub API integration supplied by Yash, AI coding agent/editor.

Yash - backend, sandbox and GitHub integration

- Own `apps/api/`, Docker configuration and GitHub App/webhook integration. Agree on run and event schemas with Durgesh and Subhi before endpoints are built.
- Create FastAPI app, Pydantic schemas, JSON/in-memory data repository, standard error format, run ownership checks and API routes.
- Implement GitHub OAuth start/callback, secure session cookie, token encryption, session middleware and logout. Keep all private keys and tokens server-side.
- Build Docker sandbox wrapper and prebuilt Python image. Install pinned dependencies before the test phase; then disable network. Use non-root, no mounts, no secrets, CPU/memory/time limits and a five-minute timeout.

- Implement `POST /webhooks/github`: verify HMAC signature, deduplicate X-GitHub-Delivery and trigger only on `issues.labeled` with label `agent-fix`.
- Integrate GitHub API actions: read issue/repo, add progress comments, push an agent branch, open a draft PR. App permissions must not allow merge.
- Implement SSE endpoint and run-event storage so Subhi can render live status updates.
- Set up smee.io or ngrok for local webhook forwarding; connect Vercel for frontend previews. Keep all secrets in local `.env` / server-only environment settings.

Tools: Python, FastAPI, Uvicorn, Pydantic, pytest, httpx TestClient, Docker / Docker Compose, GitHub App settings, GitHub REST API, smee.io or ngrok, cryptography AES-GCM, slowapi, Ruff, mypy, pip-audit, Vercel settings for frontend connection.

3. Ticket ownership map

Use the project ticket IDs already defined in 11-tickets.md. The table below assigns a clear owner; reviewers can be added on each pull request.

Person	Ticket IDs / work	Main files or outputs
You	T-17 (demo repo), T-23 QA lead, T-29, T-30, T-31, T-32, T-33; acceptance checks and deck	seeded demo repo; `tests/`; e2e review; QA checklist; demo script; release notes
Subhi	T-03 to T-05, T-09 to T-12, T-20, T-24, T-25 to T-27	`apps/web/**`; frontend tests; SEO and page content implementation
Durgesh	T-13, T-15; PR reasoning and root-cause prompts	`apps/api/pipeline/langgraph.py`; `apps/api/pipeline/llm.py`; agent prompt files
Yash	T-00 infra skeleton, T-06 to T-08, T-14, T-16, T-18, T-19, T-21, backend part of T-22, T-28	`apps/api/**`; Docker; `docker-compose.yml`; GitHub App/webhook; CI infra/deploy config

Shared tickets

T-00 / T-01 / T-02 are foundation tickets. Yash creates the repo and runtime skeleton; Subhi scaffolds the web app; Durgesh defines the agent/run contract; you check `.env.example`, README, board and acceptance criteria. One named owner creates each PR. For T-22 CI, Yash owns workflow wiring and you verify gate coverage; Subhi and Durgesh fix their own failing jobs.

4. Exact folder structure

Create one GitHub repository named `bug-fix-agent`. Keep frontend and backend separate because they are separate applications. Do not place code in random root folders.

```
bug-fix-agent/
├── apps/
│   ├── web/                    # Subhi: Next.js App Router
│   │   ├── app/
│   │   │   ├── (marketing)/        # home, how-it-works, demo, security, pricing, about, changelog
│   │   │   ├── privacy/            # privacy page
│   │   │   ├── terms/              # terms page
│   │   │   ├── auth/
│   │   │   │   ├── github/         # start sign-in
│   │   │   │   └── callback/       # callback screen / hand-off
│   │   │   ├── dashboard/
│   │   │   │   ├── page.tsx
│   │   │   │   └── runs/[runId]/page.tsx
│   │   │   ├── maintenance/
│   │   │   ├── not-found.tsx
│   │   │   ├── error.tsx
```

```
│   │   │   ├── layout.tsx
│   │   │   └── globals.css
│   │   ├── components/            # nav, footer, run-list, timeline, badges, buttons
│   │   ├── lib/                   # api-client.ts, session.ts, sse.ts, utils.ts
│   │   ├── types/                 # frontend re-exports only
│   │   ├── public/                # favicon, OG image assets
│   │   ├── e2e/                   # Playwright tests
│   │   ├── package.json
│   │   └── tsconfig.json
│   └── api/                       # Yash + Durgesh: FastAPI
│       ├── main.py
│       ├── schemas.py             # Pydantic request/response types
│       ├── webhooks/github.py
│       ├── auth/                  # oauth.py, middleware.py, crypto.py
│       ├── data/                  # repository interface + transient JSON files
│       ├── pipeline/              # langgraph.py, sandbox.py, llm.py
│       ├── sse/                   # event stream endpoint/helpers
│       ├── docker/                # Dockerfile, image config
│       ├── tests/
│       │   ├── unit/
│       │   ├── integration/
│       │   └── fixtures/
│       ├── requirements.txt
│       └── pyproject.toml
├── packages/
│   └── shared/                    # canonical Run, Step, Status, Confidence contract
│       ├── schema/                # JSON Schema shared by TS and Python
│       ├── types.ts
│       └── README.md
├── docs/                          # project specs and decisions
├── scripts/                       # check-bundle-size, demo / ops scripts
├── .github/
│   └── workflows/                 # CI checks for every pull request
├── .env.example                   # placeholders only
├── .gitignore
├── .dockerignore
├── docker-compose.yml
├── package.json                   # root scripts / workspaces if used
├── package-lock.json              # commit this if root npm is used
├── README.md
└── LICENSE (only if the team chooses one)
```

Important folder notes: the exact Next.js error-file names depend on the Next.js version. Use the App Router conventions supported by the installed version (normally `not-found.tsx` and `error.tsx`). Keep docs as source documents, but do not copy the source files into app runtime code.

Files that must never be pushed

- `.env`, `.env.*` with real values, private keys, access tokens and webhook secrets.
- `node_modules/`, `.next/`, `__pycache__/`, `.pytest_cache/`, `.venv/`, `.mypy_cache/`, coverage output, Playwright test-results and reports.
- Local run logs if they contain real issue details or tokens. Only safe demo fixtures should be committed.
- Docker build cache, local volumes, temporary clones and sandbox output folders.

```
# .gitignore essentials
.env
```

```
.env.*
!.env.example
node_modules/
.next/
.venv/
__pycache__/
.pytest_cache/
.mypy_cache/
.ruff_cache/
coverage/
playwright-report/
test-results/
apps/api/data/*.json
!apps/api/data/.gitkeep
.DS_Store
```

Environment rule

`.env.example` contains every required key with safe placeholder text only. Real values go into each developer's ignored `.env` file or a server-side deployment secret store. Never use `NEXT_PUBLIC_` for an LLM key, GitHub client secret, private key, webhook secret or session secret.

5. GitHub branch and merge rules

Use `main` as the stable branch. Nobody commits directly to `main`. Each ticket gets one short-lived branch and one pull request. This keeps work separate and lets CI find mistakes before they enter the shared build.

Branch	Who uses it	Purpose
`main`	All four; protected	Only tested, reviewed work. Deploy source.
`feat/frontend-<ticket>-<short-name>`	Subhi	Frontend page, component or integration ticket.
`feat/agent-<ticket>-<short-name>`	Durgesh	LangGraph, prompt, LLM, cache or agent logic.
`feat/infra-<ticket>-<short-name>`	Yash	API, auth, data, Docker, webhook, CI/deploy plumbing.
`feat/qa-<ticket>-<short-name>`	You	Fixtures, seeded demo, QA scripts, docs, demo materials.
`fix/<ticket>-<short-name>`	Ticket owner	Small bug found in review or testing.

Create a branch (every team member)

1. Open the GitHub repository. Confirm the issue/ticket number, owner, acceptance checks and files allowed to change.
2. Open a terminal in the repository root.
3. Get the latest `main`: `git switch main` then `git pull origin main`.
4. Create your own branch using the naming pattern above. Example for Subhi: `git switch -c feat/frontend-T09-marketing-pages`.
5. Tell the team which branch and files you are using. Do not edit another owner's files without agreement.

Work with an AI coding agent safely

1. Open the repository root in one primary tool: Codex, Open Code, Claude Code, Cursor or Anti Gravity. Use one primary agent for that ticket; do not let multiple agents edit the same working tree at once.

2. Give the agent the exact ticket, relevant `docs/` files, branch name, allowed file paths, acceptance criteria and test command.
3. Tell the agent to inspect existing code before editing. It must not rewrite unrelated files, change public API names without agreement, add a database, or invent new dependencies.
4. Ask for a short plan first. Review the plan. Then let it make one small change at a time.
5. Ask it to run tests and show the exact command and result. Read the diff yourself. The human owner is responsible for the change even when AI wrote it.
6. Never give an agent production secrets or allow it to commit `.env`. Never run untrusted code outside the sandbox.

Commit and push

```
git status
git diff --check
git diff                    # read the actual changes
# Run the ticket's tests before committing
git add <only-the-files-for-this-ticket>
git commit -m "feat(frontend): build marketing pages"
git push -u origin <your-branch>
```

Use a commit message that says what changed. Suggested prefixes: `feat`, `fix`, `test`, `docs`, `chore`. Do not use `git add .` blindly; inspect `git status` first.

Open a pull request and merge

1. Push your branch and open a PR against `main` on GitHub.
2. PR title: `[T-09] Build marketing pages`. Include ticket link, summary, files changed, test commands/results, screenshots for UI changes, known limits and any required environment variables.
3. Request a reviewer: Subhi asks you or Yash for frontend/API boundary review; Durgesh asks Yash for sandbox/API contract review; Yash asks Durgesh for pipeline integration review; you review acceptance and demo behaviour. Change reviewers if a person authored the connected code.
4. Wait for required GitHub Actions checks. Fix failures on the same branch. Push another commit; GitHub updates the same PR.
5. Reviewer checks code, security, tests and scope. Request changes when needed. Do not merge while required checks fail or while review comments remain unresolved.
6. After approval and green CI, use **Squash and merge** (recommended for one ticket = one focused history entry). Delete the feature branch after merge.
7. Everyone then runs `git switch main` and `git pull origin main` before starting the next ticket.

Protect main in GitHub settings

Set `main` as a protected branch. Require a pull request, at least one approving review, all required CI checks, and no force pushes. Disable direct pushes. Keep the GitHub App / agent without merge permission. The final PR stays draft until human review is ready; the agent must never merge.

How branches merge together

Do not create a long-lived integration branch unless the whole team explicitly needs one. Merge each finished ticket PR into `main` in dependency order. If two branches depend on a shared type or API contract, merge the contract first, then update the dependent branch from latest `main`.

```
# Update your feature branch with latest main (while on your feature branch)
git fetch origin
git merge origin/main
```

```
# Fix any conflicts manually, then run tests again
git add <resolved-files>
git commit
# Push the updated branch
git push
```

Do not use `git push --force` on shared branches. If a conflict is unclear, stop and ask the owner of the conflicting files. Never resolve a conflict by deleting another person's work without checking.

6. Correct build sequence - do not skip ahead

The project plan has phases 0–9. The safe order is below. Work in parallel only where the dependency is already satisfied. Every phase must end with its "Done when" check.

Phase	Build order	Lead owners	Exit check
0 Foundation	Repo, folder tree, Compose, `.env.example`, `.gitignore`, README, shared types, first CI workflow.	Yash + Subhi + Durgesh; you check process	`npm ci`; API and web start; `.env` ignored; CI runs.
1 Design shell	Design tokens, Tailwind, components, nav, footer, dark mode. Docker/App setup can happen in parallel.	Subhi; Yash in parallel infra	Responsive at 360/768/1280; token and accessibility checks.
2 Data + auth	Shared contract → data repo/schemas → OAuth/session → API routes. Frontend client helpers after contract is frozen.	Yash, then Subhi client libs	Unauthenticated calls 401; other-user run 404; cookie flags correct.
3 Frontend routes	Marketing pages, auth screens, dashboard, run detail, all states, SSE client.	Subhi	All required routes work; no 360px horizontal scroll; axe passes.
4 Core agent	Sandbox interface → pipeline nodes → LLM/cache → webhook → draft PR/comments; seeded repo in parallel after sandbox ready.	Durgesh + Yash; you own seeded demo	Seed issue label triggers real run; red-to-green and draft PR proof.
5 Integrate	GitHub App, webhook tunnel, Vercel preview, cross-origin SSE.	Yash + Subhi + Durgesh	Real GitHub label reaches local API; preview can show live timeline.
6 Hardening	Security headers, rate limits, all CI gates, E2E J1–J8, axe, bundle and visual baselines.	You lead QA; each owner fixes own area	All required checks green; no high/critical dependency audit issues.
7 Content + SEO	Approved copy, metadata, JSON-LD, OG, sitemap, robots, canonical, noindex dashboard.	You approve copy; Subhi implements	Lighthouse SEO target; private dashboard is noindex.
8 Launch	Production deploy, smoke test, real-phone QA, restore and rollback drills.	All four	Home 200; session endpoint healthy; mobile QA signed off.
9 Demo readiness	Monitoring, maintenance notes, rehearsed demo and recorded backup run.	You lead; others standby	2/3 seeded issues produce proof PRs; third fails honestly.

Safe parallel work

- Phase 0: Subhi can scaffold Next.js while Yash starts API/runtime folders and Durgesh drafts the shared run contract. You prepare `.env.example`, board and README acceptance checks. Merge shared types early.
- Phase 1: Subhi builds UI while Yash prepares Docker and GitHub App registration. Keep separate branches and files.
- Phase 2: Yash builds schemas/data/auth. Subhi can build frontend client helpers against the agreed contract, not guessed response shapes.
- Phase 4: Durgesh builds LangGraph nodes while Yash builds sandbox and webhook parts. They must agree on the sandbox function interface and run-state contract first.

- Phase 5: Subhi integrates SSE in frontend while Yash configures GitHub App/tunnel and Durgesh prepares cached demo responses.
- Phase 6–8: No feature expansion. Fix bugs, complete security/quality gates, then launch together.

7. Detailed task steps and tools

A. Shared contract - do this before wiring modules

1. Durgesh proposes the run state and six pipeline step names: `read`, `reproduce`, `diagnose`, `fix`, `verify`, `pr`.
2. Yash defines Pydantic models and API response shapes in `apps/api/schemas.py`.
3. Subhi checks the frontend TypeScript types match the backend fields exactly.
4. You compare the shape with `06-data-api.md` and approve it before the three developers integrate.
5. Store canonical JSON Schema / TypeScript type under `packages/shared/`. Keep Python and TypeScript field names and enum values identical.

Required status values: `pending`, `reproducing`, `fixing`, `verifying`, `completed`, `failed`, `could_not_reproduce`.
Confidence values: `high`, `medium`, `low`.

B. Yash - FastAPI, data and auth sequence

Order	Task	Tool and how to use it	Check
1	Create FastAPI app and health/docs route.	Python venv + FastAPI + Uvicorn. Start with `uvicorn main:app --reload` from the API app folder.	`http://localhost:8000/docs` opens.
2	Build JSON/in-memory repo and schemas.	Pydantic for request/response models; pytest for repo CRUD and validation. Keep repository interface small.	Unit tests cover valid/invalid records and ownership.
3	Build OAuth/session.	GitHub App OAuth; verify state; encrypt token with AES-256-GCM; `__Host-session_id` cookie: HttpOnly, Secure, SameSite=Lax, 24h.	Cookie flags and logout tests pass.
4	Add API endpoints.	FastAPI routers + `httpx.TestClient`; implement runs, run detail, SSE, retry, session, logout.	Unauthenticated=401; foreign run=404; retry requires Idempotency-Key.
5	Add rate limits.	slowapi with endpoint-specific limits from `07-security-privacy.md`.	61 requests to a 60/min endpoint return 429 with Retry-After.

C. Yash - Docker sandbox sequence

1. Create the prebuilt Docker image for the Python demo repository. Pin Python and dependency versions. Bake stable dependencies into the image to reduce demo-time network installs.
2. Implement `build_docker_cmd()` in `apps/api/pipeline/sandbox.py`. Test that the command includes `--network=none`, `--user=nonroot`, memory and CPU limits, and a hard five-minute timeout.
3. Do not mount the host filesystem. Do not pass GitHub, webhook or LLM tokens into the container. Pass only the minimum repo snapshot and issue/test metadata needed.
4. If dependency install must happen for an environment, do it before the test phase; then run tests with networking disabled.
5. Capture stdout, stderr, exit code and timeout state. Return structured results to Durgesh; do not expose secrets or raw sensitive data in logs.

6. Run an integration check that the container cannot access the network during tests and that resource/time limits are applied.

Tools: Dockerfile, Docker Compose, Docker CLI, Python Docker SDK or subprocess with explicit argument list (never `shell=True`), pytest.

D. Durgesh - LangGraph sequence

Node	What to implement	Must not continue unless
Read	Fetch issue details and check that the issue has enough expected/actual behaviour and reproduction information.	The repo is Python and issue has enough information. Otherwise comment “need more info” or “couldn’t reproduce”.
Reproduce	Ask sandbox to clone/prepare and run baseline tests; write a minimal reproduction test.	The new test fails before the patch for the expected reason.
Diagnose	Use issue + failing test + relevant source files to explain likely cause.	Cause is linked to observed test evidence.
Fix	Make a minimal patch on the agent work branch.	Patch is scoped and does not alter unrelated behaviour.
Verify	Run reproduction test after patch, then the full existing pytest suite.	Reproduction test passes and full suite stays green.
PR	Ask Yash/GitHub integration to push and open a draft PR; compose evidence and confidence.	Only after all verification checks pass. Never merge.

LLM use: call the free-tier provider through one wrapper in `llm.py`; cache by SHA-256 of issue title + body (and prompt hash for invalidation); set a 24-hour TTL; handle rate limits with bounded retry and a clear fail state. Cache rehearsed demo issues before the live demo.

E. Subhi - frontend sequence

1. Start with tokens and shell. Read `04-design-system.md`; use CSS variables for colours, type, spacing, radius and shadows. Avoid random hard-coded values.
2. Build reusable components before pages: Button, Card, Skeleton, EmptyState, Toast, StatusBadge, RunList and SSETimeline.
3. Build static marketing pages from `02-content-seo.md` and `03-app-flow.md`. Use the approved text and links. Do not claim the agent is autonomous or that it merges.
4. Build `api-client.ts` and `session.ts` against Yash’s agreed API contract. Keep browser code free of server secrets.
5. Build dashboard list and run detail. Render backend status states exactly. Display red/green evidence, root-cause text, confidence and valid PR link.
6. Build `sse.ts` with reconnect handling and de-duplication so a reconnect does not show duplicate steps.
7. Test keyboard navigation, focus rings, 44px touch targets, dark mode, loading/error/offline/empty states and mobile layout.

Tools: Next.js App Router, React, TypeScript, Tailwind, Vitest, React Testing Library, Playwright, axe-playwright, Lighthouse CI and Vercel preview deployments.

F. You - QA, demo and launch sequence

1. Create three demo issues with simple, deterministic details: (1) off-by-one in calculate_discount; (2) wrong boundary condition in validate_age; (3) missing null check in get_user_profile. Make the third issue intentionally too thin to reproduce honestly.
2. Make a QA board with columns: Backlog, Ready, In Progress, Review, QA, Done. Each ticket must include owner, branch, acceptance checks, test command and dependencies.
3. For each PR, check ticket acceptance, changed-file scope, test output and security rules. For UI changes, request screenshots at mobile and desktop widths.

4. Run the 8 E2E journeys: label-to-PR, demo flow, review PR, login/logout, failed-run recovery, IDOR negative, unauthenticated negative and SSE reconnect.
5. Run the manual QA script on Android Chrome and iOS Safari. Check 360px layout, scroll, auth, dashboard timeline, offline and error screens.
6. Rehearse the full three-issue demo. Confirm two red-to-green draft PRs and the honest failure for the third. Have a recorded fallback run ready in case free-tier LLM limits interrupt the live demo.
7. Before launch, check smoke tests, backup restore and rollback. Do not call the release complete until the checks are written down.

Tools: GitHub Projects / Issues / PR review; pytest; Playwright; axe; Lighthouse CI; Chrome/Safari devtools; physical phones; Vercel dashboard; Docker Compose; terminal; slides and screen recording.

8. Required checks before a PR can merge

Use the project's quality specification. For the hackathon, keep gates required in CI where possible. If a tool cannot be installed or run in time, record it as NOT RUN rather than pretending it passed.

Gate	Command / tool	Owner who fixes failures
TypeScript	`npx tsc --noEmit`	Subhi
Frontend lint	`npx eslint . --max-warnings=0`	Subhi
Python lint	`ruff check apps/api/`	Yash / Durgesh by file owner
Python types	`mypy --strict apps/api/`	Yash / Durgesh by file owner
Frontend unit	`npx vitest run --reporter=verbose`	Subhi
Backend unit	`pytest tests/unit/ -v`	Yash / Durgesh by file owner
Backend integration	`pytest tests/integration/ -v`	Yash
E2E	`npx playwright test --reporter=html` (Chromium + WebKit)	Subhi; you verify
Accessibility	`npx playwright test e2e/a11y/` / axe-playwright	Subhi
Lighthouse	`npx lhci autorun` on preview	Subhi; you verify
Bundle size	`bash scripts/check-bundle-size.sh`	Subhi
Node audit	`npm audit --audit-level=high`	Subhi / package owner
Python audit	`pip-audit` or project-pinned `pip audit --strict --desc`	Yash / package owner
Visual regression	`npx playwright test e2e/visual/`	Subhi; you approve baseline changes

Test order

Run the fast checks first: format/lint → type checks → unit tests → backend integration → build → E2E → accessibility/visual → Lighthouse. When a check fails, fix the cause and rerun the failed check plus the relevant earlier checks. Before final merge, run the full required CI suite.

Critical security checks

- GitHub webhook signature uses HMAC-SHA256; duplicate delivery IDs are rejected.
- Every run query checks both installation ownership and triggering GitHub login. Unknown/foreign runs return 404, not 403.
- Cookie is `__Host-session_id`, HttpOnly, Secure, SameSite=Lax, Path=/, no Domain, 24-hour expiry.
- React renders issue content and test output as text. Never use `dangerouslySetInnerHTML` for untrusted content.
- Never use `eval`, `exec`, `os.system` or shell interpolation with user-controlled strings.
- No token, issue body, PR diff, email or other personal data in logs. Keep errors generic for users.
- Docker test execution: non-root, no network after install, no host mounts, no secrets, CPU/memory/time limit.
- Agent account/App has no merge permission. Draft PR plus protected main is the human gate.

9. Tool setup and usage - simple instructions

Tool	Who	How to use it in this project
Git + GitHub	All four	Use terminal for branch/commit/push. Use GitHub Issues for tickets, PRs for review, Actions for CI, and branch protection for merge rules.
Codex / Open Code / Claude Code / Cursor / Anti Gravity	All four	Choose one main agent/editor. Open repository root; give it one ticket and allowed file paths; ask it to inspect, plan, implement, test and summarize. Review every diff yourself.
VS Code / terminal	All four	Run commands from the correct app folder. Keep one terminal per service. Check `pwd` before running setup commands.
Node.js + npm	Subhi / you for UI checks	Use the version pinned by the repo. Run `npm ci` when using the committed lockfile. Do not delete or regenerate the lockfile casually.
Python + venv	Yash / Durgesh / you	Create `.venv`; activate it; install pinned `requirements.txt`; run `pytest`. Keep Python dependencies separate from Node dependencies.
Docker / Compose	Yash; Durgesh uses interface	Build the image once; run test containers with strict limits. `docker compose up --build` starts local services. Check logs with `docker compose logs -f api` and `docker compose logs -f web`.
GitHub App	Yash	Create app for the demo repo; set callback and webhook URLs; select only needed permissions; store IDs/keys in `.env`; enable `issues` webhook. Test signature verification.
smee.io / ngrok	Yash	Start a public tunnel to local FastAPI webhook port. Copy tunnel URL into GitHub App webhook config. Keep tunnel running during live demo. Never share secrets in screenshots.
Gemini / Groq free tier	Durgesh	Store API key server-side. Put provider calls behind one `llm.py` wrapper. Add caching and graceful rate-limit failure. Prepare cached outputs for rehearsal issues.
Vercel	Yash + Subhi	Connect repository. Deploy frontend from `apps/web` (configure root directory). Preview deployments run for PRs; production deploy only from main. Set public URLs separately from secrets.
Playwright / axe / Lighthouse	Subhi + you	Run browser journeys, accessibility scans and performance/SEO checks against local or preview URL. Save test evidence to PR.

10. Common command reference

Run from repository root unless a command says otherwise. Use commands exactly as configured in the final package scripts and project files; if your team changes a command, update README and CI together.

```
# Clone once
git clone <GITHUB_REPOSITORY_URL>
cd bug-fix-agent

# Start / update branches
git switch main
git pull origin main
git switch -c feat/<area>-TNN-<short-name>

# Install and run (root scripts / workspace dependent)
```

```
npm ci
python -m venv .venv
# Windows: .venv\Scripts\activate
# macOS/Linux: source .venv/bin/activate
pip install -r apps/api/requirements.txt

docker compose up --build

# Checks
git status
git diff --check
npx tsc --noEmit
npx eslint . --max-warnings=0
ruff check apps/api/
mypy --strict apps/api/
npx vitest run --reporter=verbose
pytest apps/api/tests/unit/ -v
pytest apps/api/tests/integration/ -v
npx playwright test --reporter=html
npx lhci autorun

# Commit and publish
git add <ticket-files>
git commit -m "feat(area): short description"
git push -u origin <your-branch>

# After the PR is merged
git switch main
git pull origin main
```

Important command note

The plan mentions `npm ci && npm run check` and `docker compose up`. Make sure the root `package.json` actually defines `check` and that the root lockfile exists before relying on those commands. The Python install path should be documented in README and match the real requirements file.

11. Decisions and source-document notes

Topic	Safe rule for this sprint
Database	Do not add PostgreSQL/Supabase. Architecture ADR-006 says no database in v1; use JSON files / in-memory behind a repository interface. The SQL schema in the data/API document is a future data contract, not a live DB requirement.
Demo scope	Python + pytest only, our seeded repo only. Other languages are not part of v1.
Human approval	Agent opens draft PR only. A human approves and merges. No auto-merge or agent merge permission.
Frontend route count	Build the pages listed in the product brief and app flow. Use the installed Next.js App Router file naming rules.
SSE	FastAPI sends one-way live events to Next.js. Do not introduce WebSockets for this sprint.
Run storage	Transient JSON/in-memory data; do not store persistent customer issue data for the demo.
Free LLM limits	Use cache for rehearsed issues and have a recorded run. Never fake an API result or test pass.

If project files disagree, pause and resolve the decision in the relevant `docs/` file before changing code. Keep the architecture, API contract and frontend types in sync. A decision change should be documented first, then implemented by a small ticket.

12. Daily team routine and definition of done

Start of work

1. Check the project board. Each person takes one ticket at a time.
2. Read the ticket and its referenced docs. Confirm dependencies are merged.
3. Create a feature branch from the latest `main`.
4. Tell the team the ticket, branch and files you will touch.

During work

1. Use one primary AI agent for one ticket. Keep prompts limited to this ticket.
2. Make small edits. Run the smallest relevant test after each change.
3. Do not change contracts, folder structure or dependency versions without agreement.
4. Commit only the files for the ticket. Push before requesting review.

Before marking a ticket Done

1. Acceptance checklist is complete.
2. Tests were run and real outputs are visible in the PR.
3. Diff is reviewed; no secrets, generated files or unrelated edits are present.
4. Required CI checks are green.
5. At least one human approves. PR is merged to main.
6. Ticket is moved to Done and the branch is deleted.

Final project Definition of Done

The demo repo receives the `agent-fix` label on three prepared issues. Two or more runs open draft PRs with true failing-before / passing-after proof, full-suite green status and root-cause notes. The third run posts an honest “couldn’t reproduce” comment. The dashboard shows every run. Docker isolation is active. Branch protection requires human approval. The AI agent cannot merge.

Project files used to prepare this guide

- 00-brief.md - product idea, non-negotiable features, scope and demo definition
- 01-prd.md - MVP requirements, user stories, non-goals and risks
- 02-content-seo.md - page copy, voice and SEO details
- 03-app-flow.md - routes, journeys, states and auth flow
- 04-design-system.md - UI tokens, components and responsive rules
- 05-architecture.md - stack, folder tree, environment and module boundaries
- 06-data-api.md - data contract, endpoints, events and errors
- 07-security-privacy.md - threat model, auth, access, sandbox and secrets
- 08-quality.md - unit, integration, E2E, accessibility and CI gates
- 09-operations.md - deploy, smoke tests, backup, rollback and maintenance
- 10-plan.md - phases, dependencies and parallel work
- 11-tickets.md - small build tickets, acceptance checks and dependencies